Multivariate Long Term Energy Time Series Forecasting



SAPIENZA
UNIVERSITÀ DI ROMA

References

- **Student:** Tacconi Francesco
- **Student ID:** 1397715
- **Thesis Title:** *Long Term Multivariate Energy Time Series Forecasting*
- **Supervisor:** Panella Massimo
- **Co-supervisor:** Verdone Alessio
- **Degree Course:** Facoltà di Ingegneria dell'Informazione, Informatica e Statistica - Corso di Laurea Magistrale in *Ingegneria Gestionale*
- **Academic Year**: 2025-2026

Why Energy Forecasting?

- **Grid Stability :** Maintaining a constant balance between supply and demand to ensure the "smooth flow" of electricity without structural overloads.

- **Smart Procurement:** Buying energy in advance at lower prices, avoiding expensive "last-minute" coal or gas plants (Peaking Plants).

- **Resource Allocation:** Optimizing the distribution of power across the network based on predicted long-term usage patterns.

- **Demand Response:** Offering discounts to major customers to shift their consumption to off-peak hours.

- **Risk Mitigation:** Identifying potential imbalances or equipment stress before they lead to blackouts or infrastructure damage.

Case Studies: ECL vs. ETTh1

1. Electricity Consuming Load (ECL)

- **Plan:** Monitoring the electricity consumption (kW) of 321 individual customers to map the "energy habits" of a specific region.
- **Grid Management:** Utilizing accurate forecasts to ensure the grid is prepared for collective demand spikes, such as morning or evening peaks.

2. Electricity Transformer Temperature (ETTh1)

- **Asset Health Monitoring:** Predicting Oil Temperature (OT) as the primary indicator of a power transformer's health.
- **Preventive Maintenance:** Extending the asset's lifespan by detecting and preventing hazardous operating conditions (e.g., overheating) before they occur.

Data: Benchmark Energy Datasets

Datasets	ETTh1 & ETTh2	ETTm1 & ETTm2	Electricity (ECL)
Variates	7	7	321
Timesteps	17,420	69,680	26,304
Granularity	1 hour	15 min	1 hour
Domain	Power Transformer	Power Transformer	Consumer Load

1. Electricity Consuming Load (ECL)

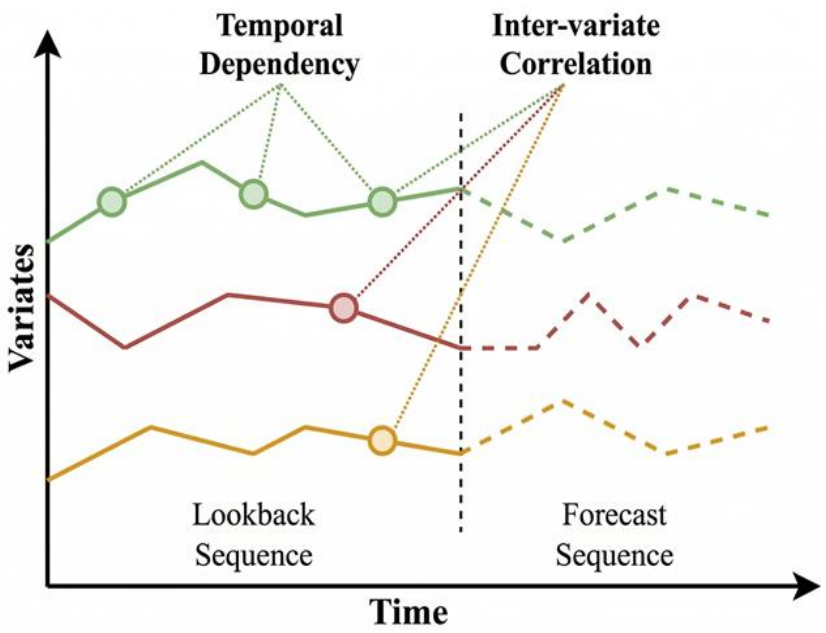
- *source: Trindade, A. (2015). ElectricityLoadDiagrams20112014 [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C58C86.*
- electricity consumption [kW] of 321 individual clients in Portugal between 2011 and 2014

2. Electricity Transformer Temperature (ETTh1)

- *source: https://github.com/zhouhaoyi/ETDataset*
- Oil temperature [°C] and 6 Loads [kW] for a given power transformer in China from July 2016 to July 2018.

Task: Multivariate Time Series Forecasting

Temporal dependencies (**TD**) and inter-variate correlation (**VC**)



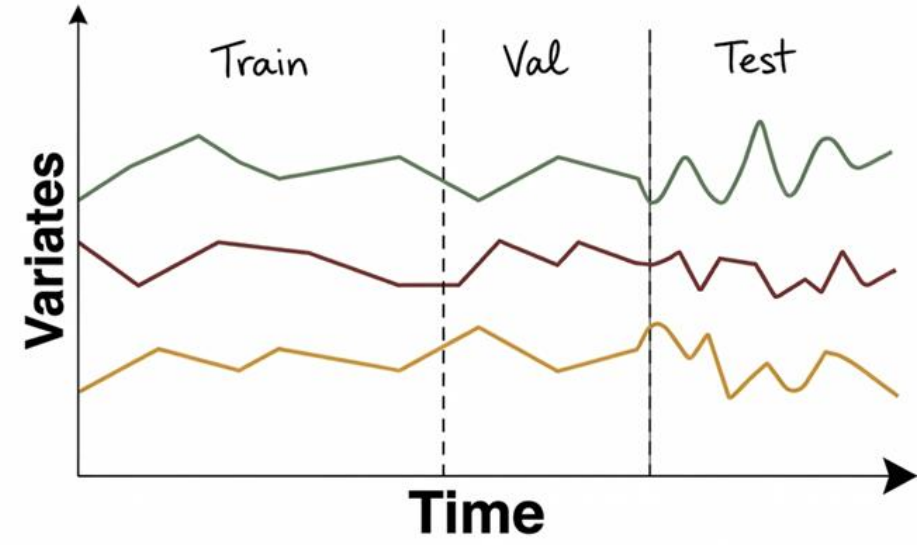
Goal of TSF:

- $given\ \boldsymbol{X} \in \mathbb{R}^{L \times V}$
- $Predict\ \boldsymbol{Y} \in \mathbb{R}^{T \times V}$

Supervised Problem

- train/val/test split
- Use “train set” to make model learn
- Use “val set” to early stop the model
- Use “test set” on truly *unseen* data

Data Prep: Splits and Normalization



Data *Splits*:
(ts = time steps)
- **Train** (70% ts)
- **Validation**(15% ts)
- **Test** (15 % ts)

In this example we have V=3 channels (variates)

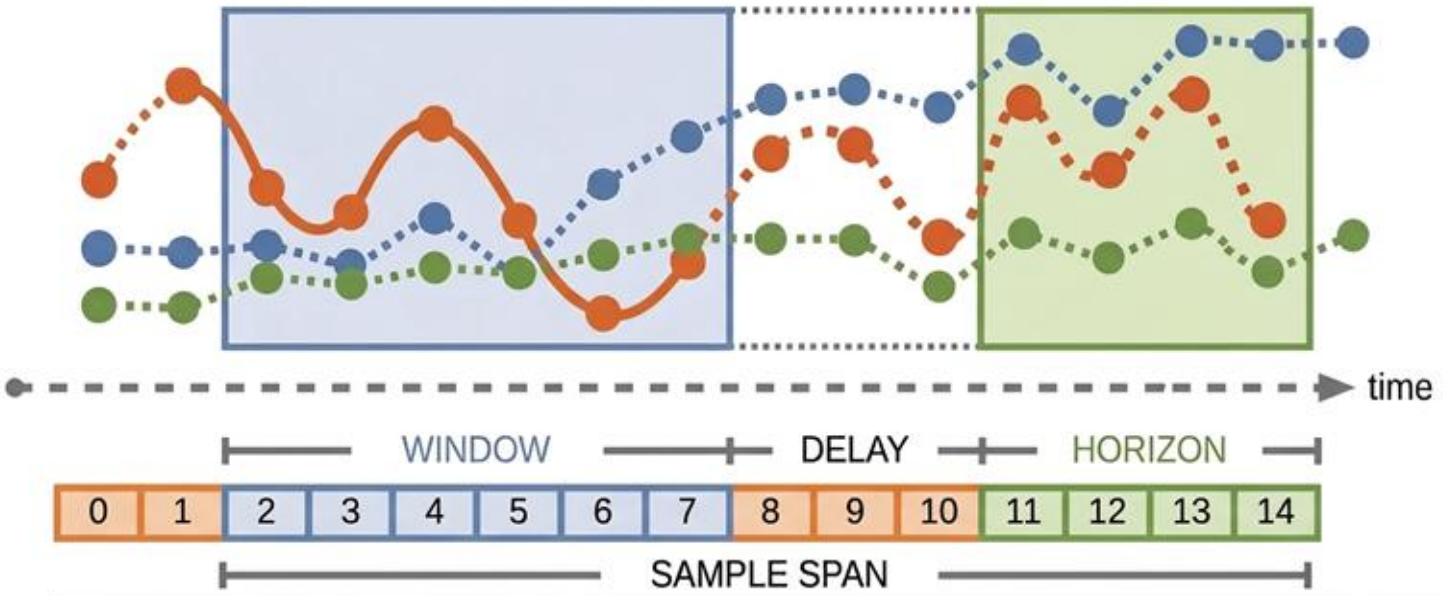
Given V channels over N_{tot} discrete time steps, we apply ***per-channel z-score normalization***. For each channel v, $\boldsymbol{\mu}_v^{\text{train}}$ and $\boldsymbol{\sigma}_v^{\text{train}}$ are computed on the $N_{\text{tot}}^{\text{train}}$ training time steps, and

$$\tilde{x}_t^{\,v} = \frac{x_t^{\,v} - \boldsymbol{\mu}_v^{\text{train}}}{\boldsymbol{\sigma}_v^{\text{train}}}.$$

Validation and test are normalized using the same training statistics to prevent data leakage.

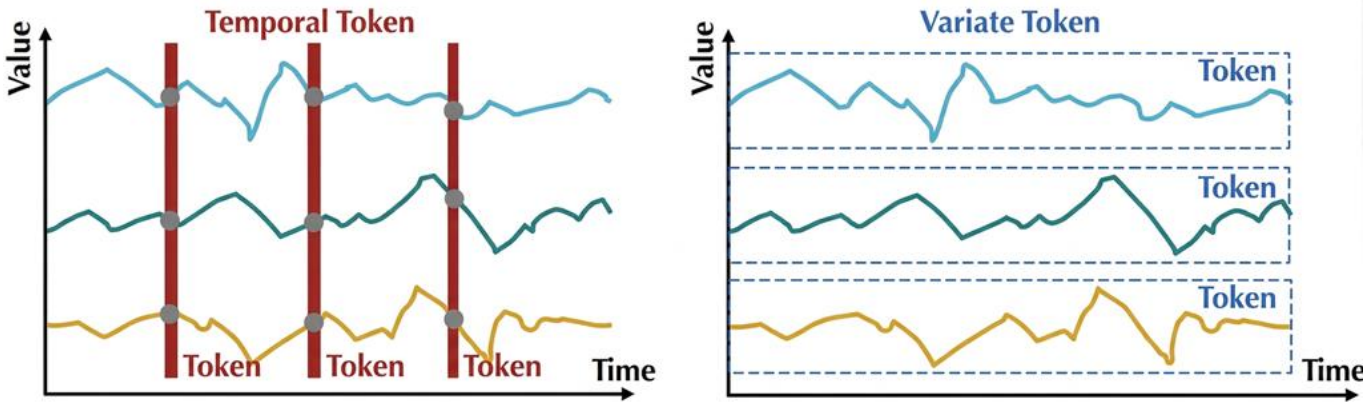
Dataset creation: data windowization

Shape data in correct format to *feed* the model



SUMMARY: For each split: Stride=1 (max overlap), Delay=0, Window=L; Horizon=T.
Each windowization forms a supervised dataset row: $(\boldsymbol{X} \in \mathbb{R}^{L \times V}, \boldsymbol{Y} \in \mathbb{R}^{T \times V})$.

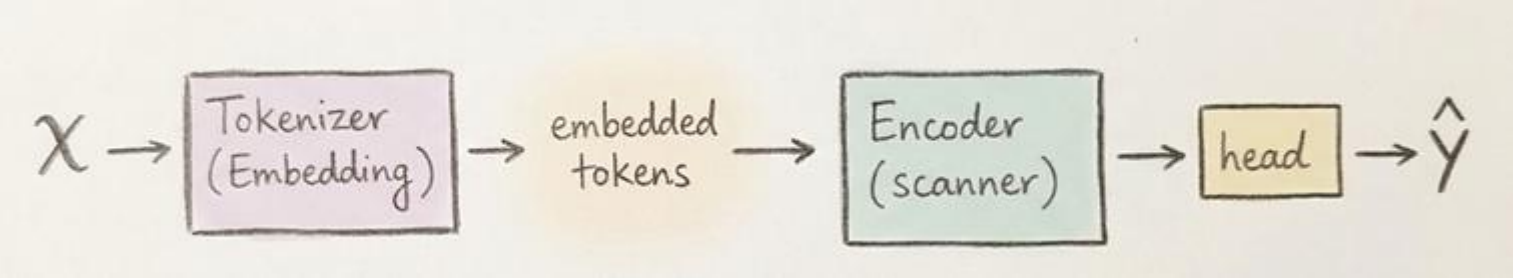
Tokenization: Time Tokens and Variate Tokens



- *Input*: $\boldsymbol{X} \in \mathbb{R}^{L \times V}$
- **Temporal Tokens**:
  - L tokens of dimension V
  - Limitation: High noise; points lacks semantic meaning
- **Variate Tokens**:
  - V tokens of dimension L
  - Logic: Flips the axis; each token is a full channel history
- *The "Scan" Philosophy*:
  - Goal: Extract relations across tokens (both TD and VC)
  - Temporal View: Focused on Time-Domain (TD) dependencies
  - Variate View: Focused on Variate-Centric (VC) correlations

Models Backbone: the general structure

Structure shared across all *experimental models*



Principal layers:

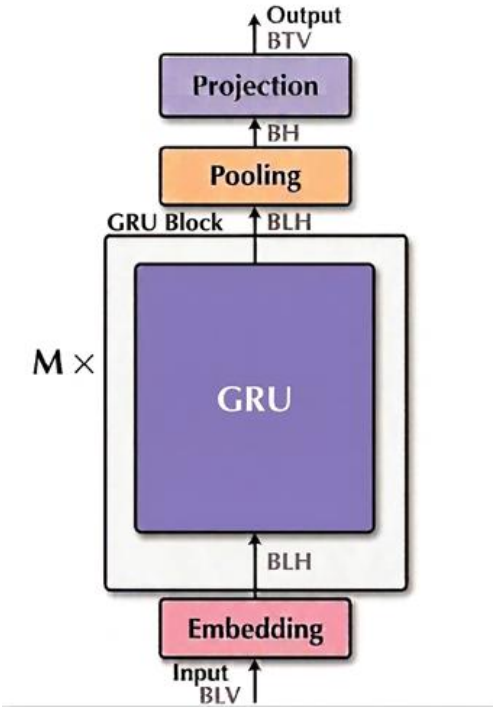
- **Tokenizer:** linear projection to increase token dimension
- **Encoder:** the "token scanner", and conceptual *core* of the model
- **Head:** linear projection of "context-aware encoder output" to prediction $\hat{y}$

Additional notes:

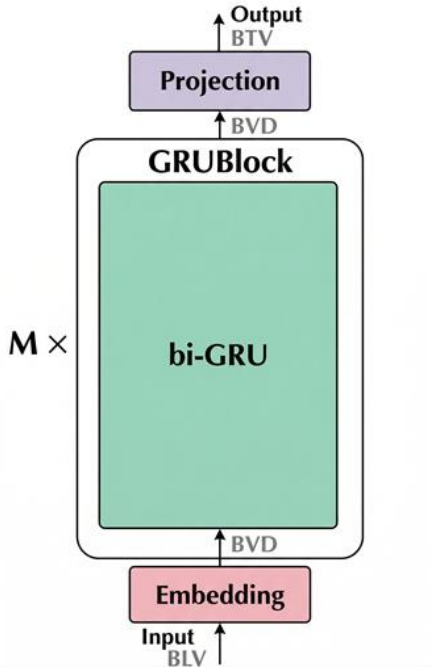
- Params update happens through $MSE(Y, \hat{Y})$ Loss
- In "time setting" we add *pooling* (additive or last) to encoder-output, to collapse the L-dim and avoid parameter explosion
- In time setting with Transformer encoder, we add *Positional Encoding* to Tokenizer
- In "variate setting", Tokenizer "flips" the input

Model type 1: GRU

GRU - Time Setting



GRU - Variate Setting



Legend:
- N = number of tokens scanned
- d = embedded token dim

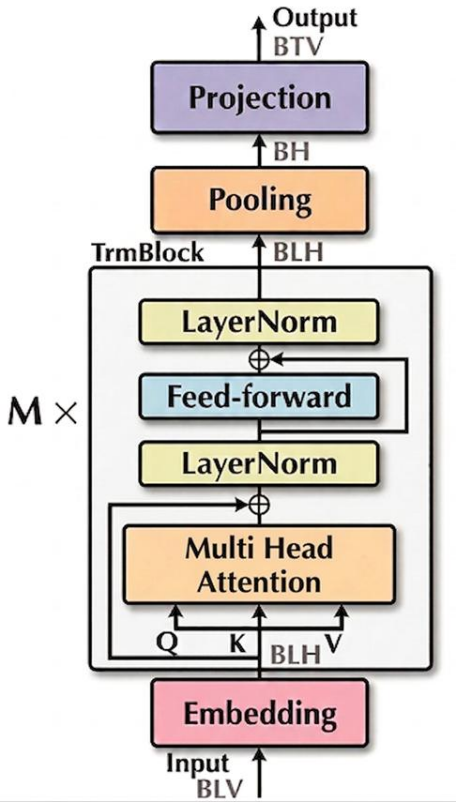
Architecture:
- Stackable blocks(dim. preserving)
- No “add & norm” wrap

Notes on “*Variate scenario*”:
- does not need pooling
- *Bidirectional*, because variate tokens does not have a “temporal order”

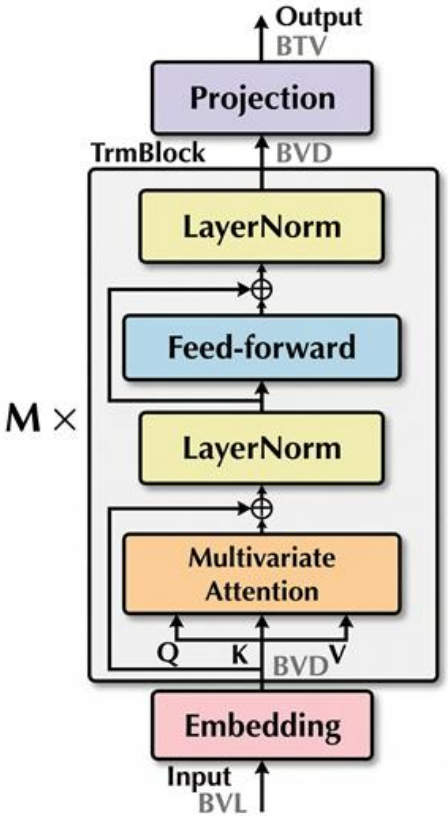
GRU Encoder:
- **Scan type**: recurrent
- **Cost:** $O(Nd^2)$

Model type 2: vanilla Transformer

Transformer – Time



Transformer – Variate



Legend:
- N = num tokens
- d = embedded token dimension
- d_{ff} = dim feed-forward
- σ = relu / gelu

Architecture:
- "**stackable blocks**", because are input/output dimension preserving
- ***Add & Norm*** "wrap", to stabilize the scan info extraction

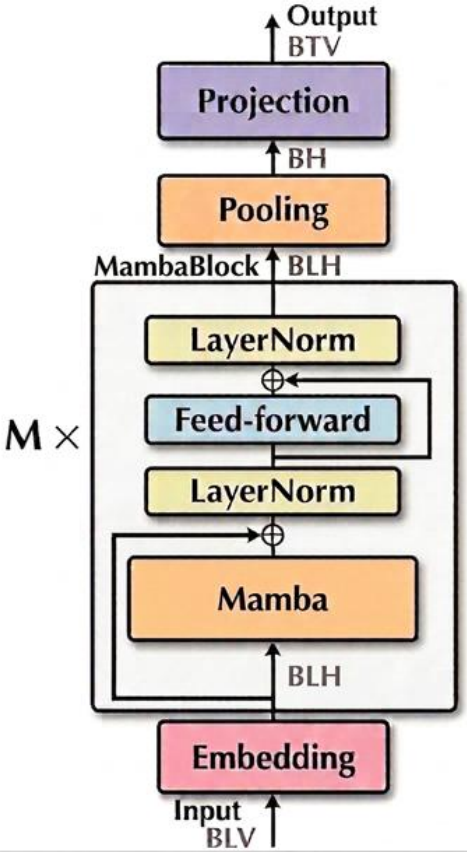
Variate Notes:
- Does not need PE (channels not sequentially ordered)
- Self attention better suited to handle "variate tokens

Self Attention:
- **Scan type**: parallel scan
- **Cost**: $O(N^2\, d)$, quadratic over N

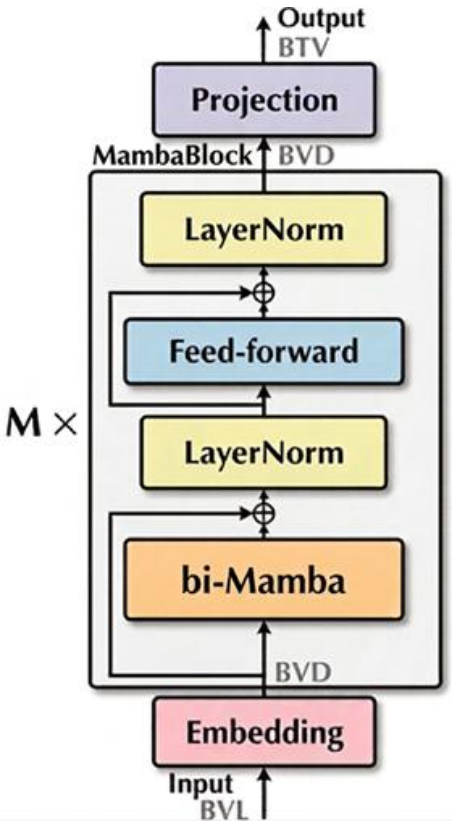
FFN: $\sigma(W_1 x)W_2$
- proj1($d \to d_{ff}$), act, proj2($d_{ff} \to d$)
- **cost:** $O(N \cdot d \cdot d_{ff})$

Model type 3: Mamba

Mamba -Time



Mamba - Variate



Architecture:
- Still “Add & Norm” wrap
- Still “Stackable Blocks”

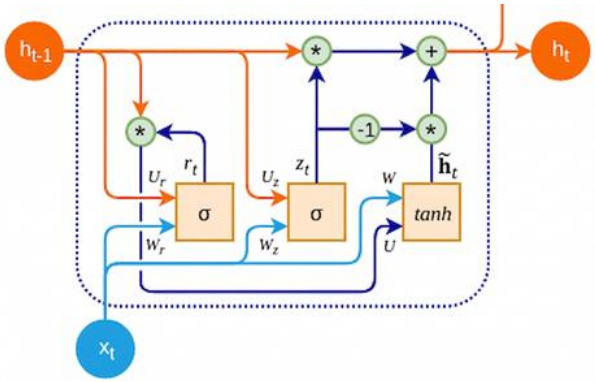
Variate notes:
- Using “Bi-Mamba” to “scan channels interaction” from both sequential order (self attention surrogate)

SSM (s6 kernel):
- **Scan type**: selective scan (*recurrent* but optimized on GPU cuda kernels)
- **Cost**: O(Nd) , highly optimized

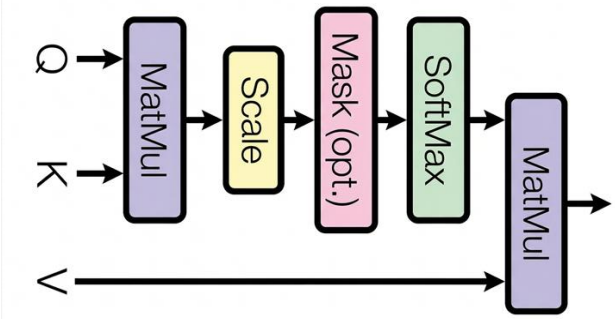
Inside the “black box”: the scanners

A “glimpse” within the encoder TD and VC token infos extractors

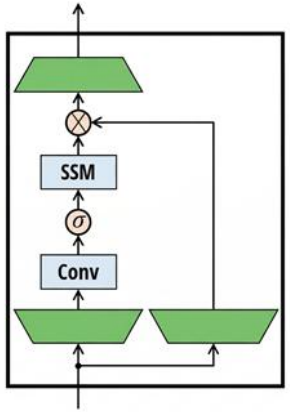
GRU



Dot product attention



Mamba block



Mamba

$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z)$
$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r)$
$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h)$
$h_t = (1 - z_t) \odot \tilde{h}_{t-1} + z_t \odot \tilde{h}_t$
z_t, r_t= *update gate, reset gate*
$h_t = memory\ state$

Considerations:

- Inherently slower due to sequential/recurrent processing.
- Susceptible to vanishing gradients during Backpropagation Through Time (BPTT).

*** learnable params*

$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$

$$Attn(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d}}\right)V$$

Given *N tokens* each having $d = d_{model}$, those are the shapes:
$X \in \mathbb{R}^{Nd}$, $W \in \mathbb{R}^{dd}$, so
$Q, K, V \in \mathbb{R}^{Nd}, (*) \in \mathbb{R}^{NN}$ *and* $attn(*) \in \mathbb{R}^{Nd}$

Considerations:

- Global context (no internal memo: sees all tokens in parallel)
- But quadratic cost over N

SSM:

- continuous system *discretization:*
  $\bar{A} = \exp(\Delta A), \bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I)\Delta B$
- *Selective (input-dep) params:*
  $B_t = s_B(x_t), C_t = s_C(x_t), \Delta_t = \tau(S_\Delta(x_t))$

Where s(*)=linear, $\tau(*) = sofplus$

- Discrete s6 kernel:

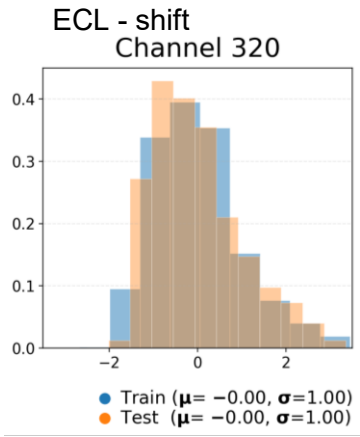
Considerations:

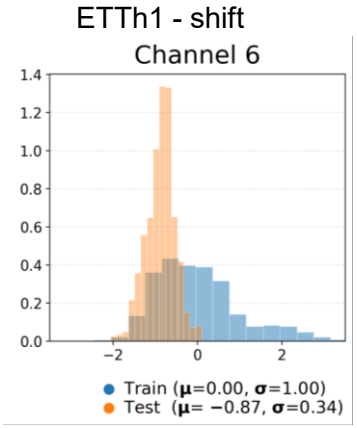
- Recurrent (internal state update) like RNNs
- Selective scan able to forget/update info
- Even being recurrent, has long context and is efficient

Experiment Results: Test Scores

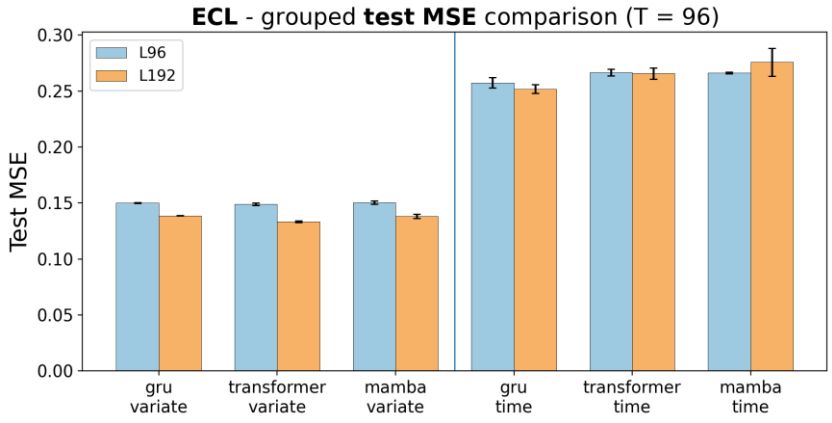
TEST METRICS (T=96)

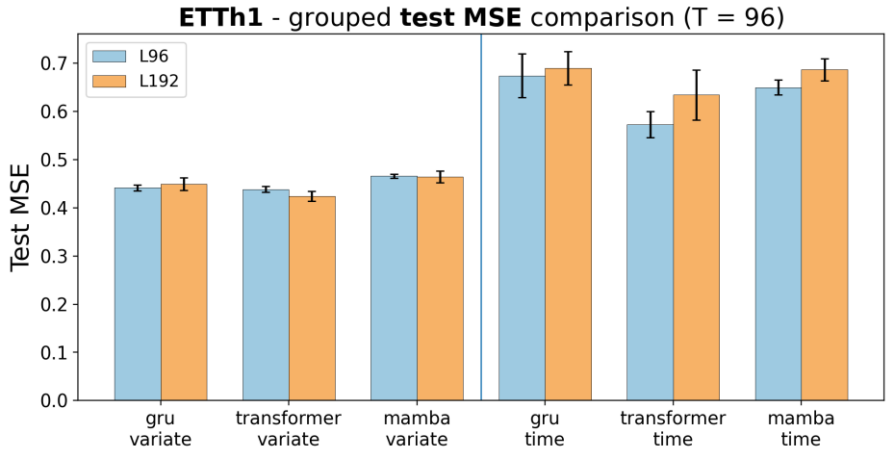
	MSE(L=96)	MSE(L=192)
ECL		
gru-variate	**0.1502 ± 0.0004**	0.1388 ± 0.0003
transformer-variate	**0.1489 ± 0.0011**	**0.1335 ± 0.0008**
mamba-variate	0.1505 ± 0.0014	**0.1382 ± 0.0019**
gru-time	0.2574 ± 0.0048	0.2518 ± 0.0038
transformer-time	0.2667 ± 0.0030	0.2657 ± 0.0051
mamba-time	0.2663 ± 0.0008	0.2757 ± 0.0125
ETTh1		
gru-variate	0.4415 ± 0.0063	0.4490 ± 0.0132
transformer-variate	**0.4384 ± 0.0061**	**0.4241 ± 0.0100**
mamba-variate	0.4658 ± 0.0044	0.4645 ± 0.0121
gru-time	0.6741 ± 0.0455	0.6896 ± 0.0347
transformer-time	0.5727 ± 0.0271	0.6343 ± 0.0520
mamba-time	0.6499 ± 0.0151	0.6865 ± 0.0230



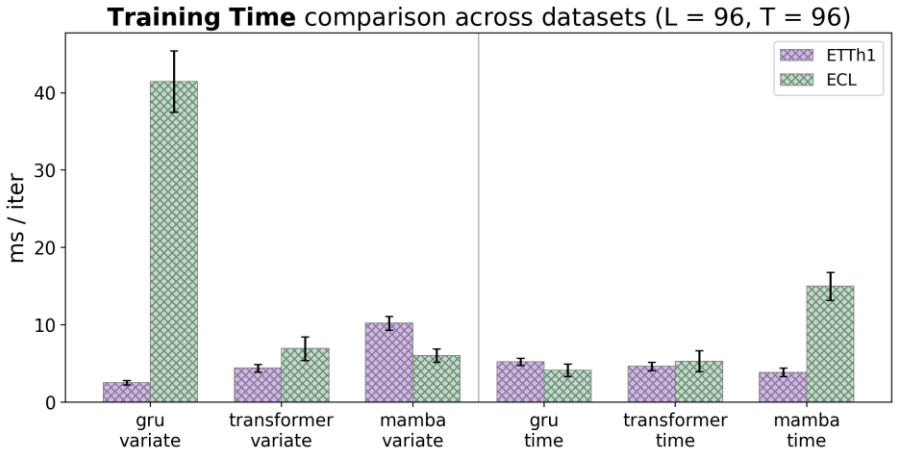


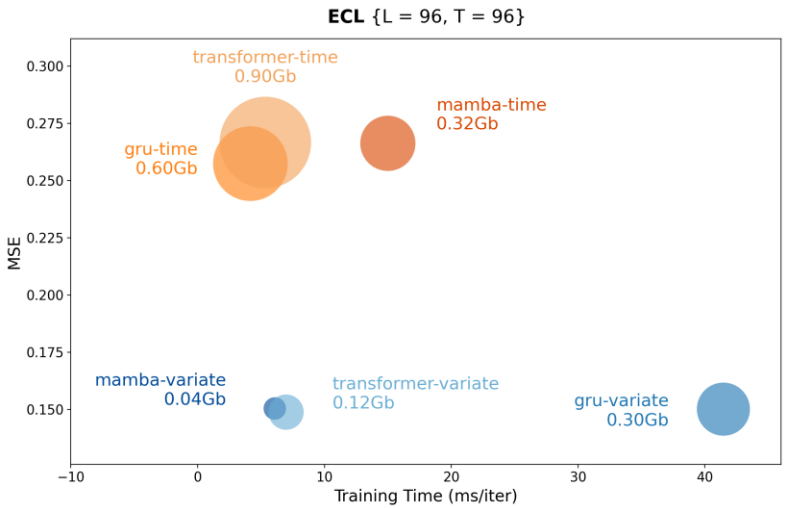
$$MSE = \frac{1}{\bar{N}}\sum_{i=1}^{\bar{N}}\left(Y-\hat{Y}\right)^2, \qquad \bar{N} = test\ timesteps$$





Exp. Results: Error, Training Time and Size





- ***Training Time*** *(ms/iter) = full fwd & bwd pass thru a batch of 32 samples (GD and weights updates)*
- ***Size*** *= GPU-memo consumption [Gb], correlated with model number of params [Mb]*

ECL
- Gru-variate slow -> not only for N=321 (in place of 96), but also because tuned model is "heavy" (*large* d_{model}, n_{layers} , *bidirectionality*)
- Mamba-time slow too (same reason)
- Mamba-variate and transformer-variate much *faster* than time counterparts (less noisy tokens, tuned models lighter)

ETTh1 -> N=7 small, d and d_{ff} dominating cost factors
- Mamba-variate suffer from "short scan" more than self attention
- Gru-variate benefits from N=7 and FFN absence
- Gru-time suffer passing from N=7 to 96

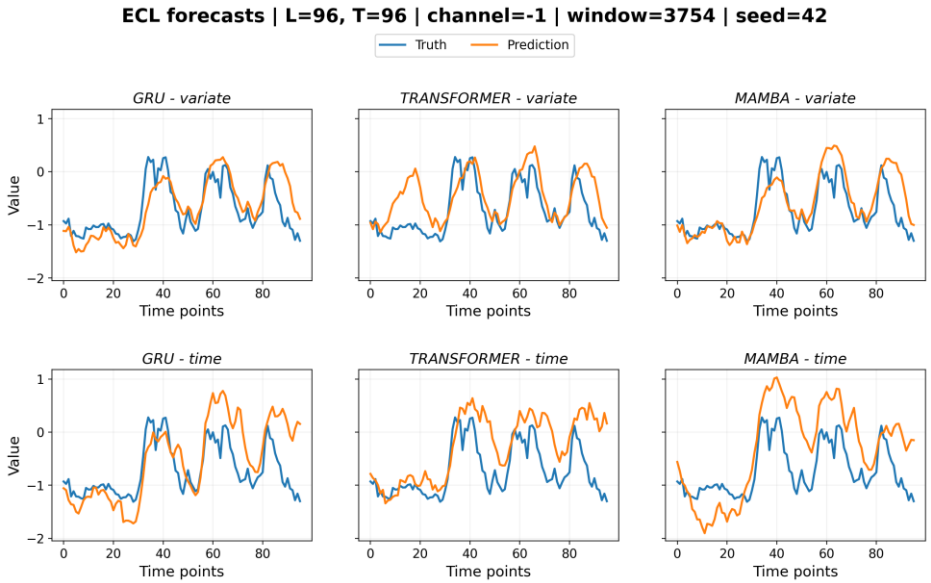
Recap: Training time deps on N, d_{model} and scan mechanism

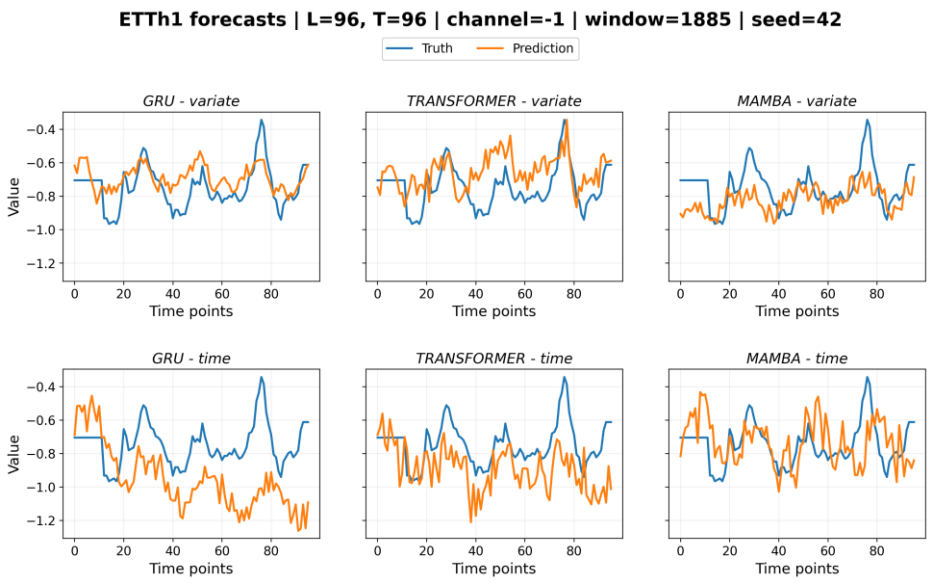
1. Size vs Training Time -> no direct correlation
- *Transformer-time* is big but quite fast (parallel computation)
- *Gru-variate* smaller but slow (process 321 tokens sequentially)

2. GPU memo vs Tokens -> correlated
- Time models are bigger even having less tokens, because those are noisier and needs bigger hparams
- Gru-variate smaller than Gru-time, but slower (scans more tokens)

3. Error vs Training Time
- Variate models better (lower MSE, comparable time, less memo)
- Transformer-variate best
- Mamba-variate *best overall* (similar MSE, less memo, faster)
- Gru-variate remains accurate, but really slow

Exp. Results: Trajectories Analysis





ECL

- *Variate*: smoother, aligned and correct peaks
- *Time*: overestimates peaks, phase shift, noisier trajectories

ETTh1

More challenging (preds less accurate)

- Still variates better
- Mamba-variate captures trend, but underestimate peaks (maybe N=7 is too short for “selective scan”)
- Transformer-variate less sensitive to short sequence, and understands peaks

Conclusions and Final Remarks

1. **Main Thesis Contribution**: *Superiority of Variate Tokens*

- Scrutinizing inter-variate correlations (VC) consistently outperforms time-domain (TD) scanning.
- Variate-centric models produce smoother, better-aligned trajectories and are more robust against noise.

2. Most Promising Architectures:

Transformer-variate and *Mamba-variate* emerged as the top-performing models across both benchmarks.

3. The "Mamba" Efficiency: *Memory Footprint*

Mamba-variate is the most efficient model in terms of GPU memory consumption, offering a significantly lower footprint than Transformers.

- **High-Channel Scaling:** It excels in large-scale scenarios (ECL) where its selective scan mechanism handles long variable sequences with high speed.

4. Operational Trade-offs:

- **GRU Limitations:** While accurate, GRU-variate is penalized by its sequential nature, resulting in high latency as the number of channels increases.
- **Risk Assessment (ETTh1):** Mamba tends to underestimate peaks in low-channel datasets. For critical assets like transformers, this poses a risk of missing Oil Temperature spikes.